n = 19 tau6: the per-shift Groebner solve moved out of Sage, end-to-end cold

2026-10-19

n = 19 tau6: the per-shift Groebner solve moved out of Sage, end-to-end cold

Status (2026-10-19): protocol frozen; the Tier 1 run is pending. The protocol was frozen at 2026-10-19T19:00:03Z, before any timing. The schedule and the held-out panel were committed with it. Everything below that is not quoted from the archived run 38520305068 is a **local diagnostic** from the cloud-agent VM. These diagnostics are counts and exactness checks, not timing evidence. See `smoke/`.

Question

The n = 19 row of `docs/ic/BASELINES.md` is the volcano-ic τ-orbit dim-6 base, arm `tau6`. Its numbers:

- run 38520305068;
- protocol `../volcano-ic/rho-arm-20261014/protocol.json`;
- cryptanalysis `3f722e2e`;
- cold `S` = 5,154, and online `S` is "not separable";
- cold gap to the matched rho: 1.94e4 [1.69e4, 2.23e4].

The row's next action is "move tau6's per-relation Gröbner solve out of Sage".

This experiment asks: does replacing that solve with a native enumeration of the same variety lower the **end-to-end cold** CPU cost of one complete, verified ECDLP? Every target, every shift system, every relation and every log must stay unchanged. This is an iteration under AGENTS.md "Success criterion". If it is promoted, it becomes the new n = 19 row.

Where the baseline's time goes (archived run; cited)

Sources: `../volcano-ic/rho-arm-20261014/results/arms.jsonl`, the ten `tau6` rows, and `results/runtime-info.json`. Both come from run 38520305068 on an AMD EPYC 9V45 GitHub VM, using the `sagemath/sagemath:10.10` image.

phase (tau6, 10 complete ECDLPs)	CPU s	share
per-shift solve (`gb_cpu_s`: `TauField.system` + `ideal` + `groebner_basis` + `variety`)	402.147	**99.39%**
rest of the relation search (targets `aP + bQ`, shuffles, base and point checks)	2.421	0.60%
linear algebra (sparse left kernel mod r)	0.025	0.006%
total (`total_cpu_s`)	404.593	100%

phase (tau6, 10 complete ECDLPs)	CPU s	share
setup (TauField, TauCurve, generator; all three arms, outside every interval)	0.372	not charged

- The 10 ECDLPs made 54,336 solve calls, **7.40 ms** each. They drew 3,057 targets and collected 39 relations per ECDLP (29 orbit columns + 10).
- **There is no rank stage and no reusable table.** Every relation is R = aP + bQ, so it contains the target, and the whole pipeline is one-target. This is why the row says online `S` is "not separable": cold and online differ only by the target-independent setup. That setup was 0.37 s for all arms together and was not timed per arm.
- **The Sage solve dominates.** The lever is the per-shift solve, which matches the row's stated next action.

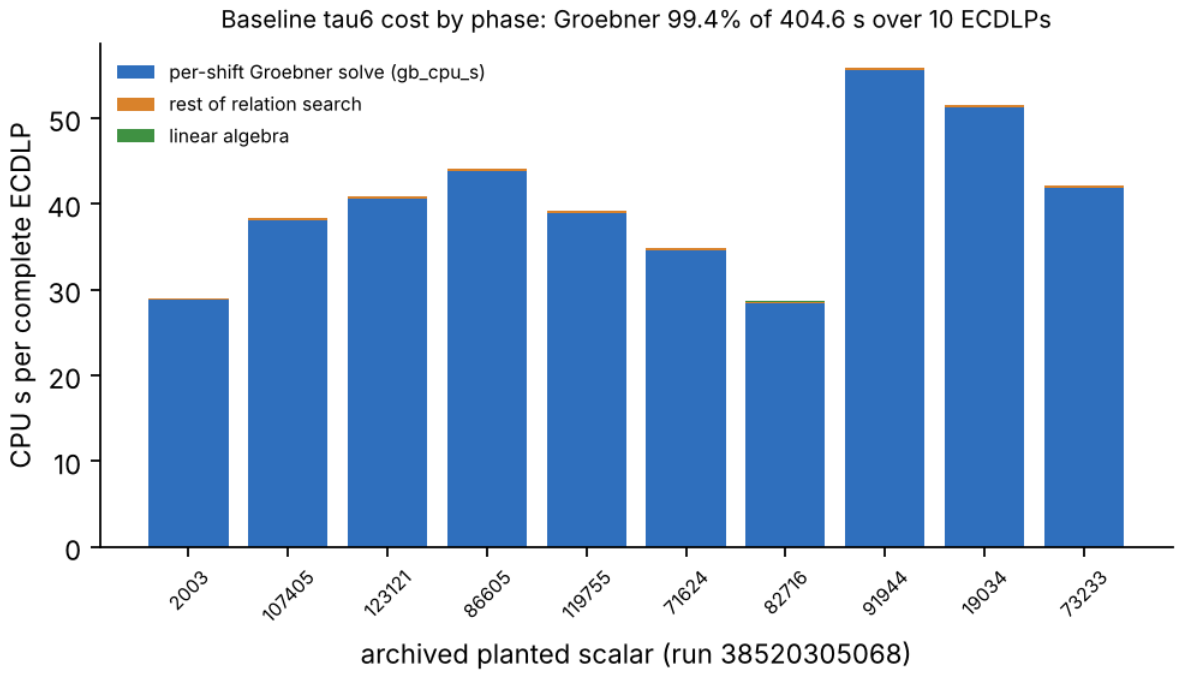


Figure 1: Baseline phases per archived ECDLP (run 38520305068). Source: make_figures.py `design`; vector: figures/baseline_phases.svg.

The change

The solve now runs in `native/src/lib.rs`, a Rust `cdylib` with no dependencies, loaded through `ctypes` by `arm.sage`.

- **What stays.** The candidate calls `tau.sage`'s own `tauDlp` unchanged: rng, targets, shift order, matrix, left kernel and verification. Only the global `tauRelation` is rebound to `tauRelationNative`. Its loop is `tauRelation` line for line once the variety is known: the V′-base membership, the four sign checks ±`P1` ± τ`^j P2 == R` in Sage, and the returned `(column, sign, Frobenius power)` terms.
- **Setup.** The candidate builds `NativeTauField`, which has no Boolean ring and no shift systems, then `TauCurve` and the native context.

Why it is exact. `TauField.system(j, xR, b)` is, coordinate by coordinate over GF(2^19), the third summation polynomial

S3(x1, x2, xR) = (x1 x2)^2 + xR^2 (x1^2 + x2^2) + xR x1 x2 + b,

with x1 = Σ u_i z^i and x2 = σ^{j}(Σ w_l z^l). Its variety over F_2 (the Boolean ring adds the field equations) is the set of (u, w) ∈ F_2^6 × F_2^6 with S3 = 0.

The native code decides every one of the 4,096 pairs:

- For fixed u, S3 = a x2^2 + c x2 + d with a = (x1 + xR)^2, c = xR x1 and d = c^2 + b. The map x2 ↦ a x2^2 + c x2 is F_2-linear, so the condition is Σ_l w_l g_l = d with g_l = a Y_l^2 + c Y_l, where Y_l = $\sigma^{j(z}$l).
- a, c and d are affine in the bits of u, so 49 field multiplications per call build their increments. A Gray-code walk over u, and over w inside it, then checks every pair with XORs.

Order of solutions. The variety is a set. The order in which `tauRelation` walks it matters only when a system has two or more solutions. Sage's order is not specified, and the native code returns ascending (u, w).

- **Over all inputs.** An exhaustive local survey of every (xR, j) found 2,375 such systems among 9,961,472 (`smoke/multi_survey.json`). 2,001 are the shift-0 swap (a, b), (b, a). 374 are not: 4 at shift 0 and 370 at shifts 1 to 18.
- **On this panel and the archived scalars.** The candidate meets 26 multi-solution systems, and all 26 are shift-0 swaps (`smoke/multi_calls.json`).
- **Why swaps are harmless.** Both members of a swap are in the base or neither is, and both are decompositions of R or neither is. The order therefore cannot change `spurious`, the relation found or its matrix row. It can only swap the two terms of the returned decomposition.
- **What the gates compare.** Matrix rows and decompositions as multisets across B and N, with the ordered lists recorded. The ordered lists must be identical across N, Naa and NX.

Local exactness and diagnostics (cloud-agent VM; not timing evidence)

These checks used only the cloud-agent VM. Sage was not available there.

- **Native tests.** `cargo test --release` passes 3 tests. On 400 random (xR, j), the Gray-code solve equals the direct per-u form and a brute-force S3 evaluation, and the four known two-solution systems come out in (u, w) order.
- **Archived replay.** `smoke/local_replay.py` redoes `tauDlp`'s control flow in pure Python, using `rho_frob.py`'s PyBackend arithmetic and the native solve. It uses the archived generator, λ and rng seeds. On all ten archived scalars it reproduces run 38520305068's tau6 `targets`, `gb_calls`, `spurious`, `relations` and `log` exactly (`smoke/local_replay_archived.json`). For the first 300 systems it also checks the native variety against an independent pure-Python S3 evaluation of all 4,096 pairs.
- **Held-out panel.** `smoke/local_panel.py` runs the same check on the 16 panel points. All logs verify, with a mean of 5,213 solve calls per ECDLP (`smoke/local_panel.json`).
- **Per-call cost.** The native solve takes about 14 μs per call, including the ctypes call, against 7.40 ms per Sage call in the archived run. Both numbers are on different hosts and only indicate size; the run decides.

Two gates check the Sage side on the runner itself:

- S0: Sage variety equals the native variety on 300 systems;
- NX: every shift system of every panel point is compared with Sage.

Online `S`: now separable, by definition and by measurement

The pipeline has no reusable factor-log table, so "online" cannot mean "with a cached table". The protocol therefore splits every process into two intervals, both timed with `time.process_time()`:

- **setup**: target-independent construction (field, `TauCurve` base and orbit representatives, native context, generator);
- **online**: `tauDlp`, from the first target-dependent operation to the verified log.

Cold is setup + online. Each row also records the clock resolution.

The archived run timed setup once, for all arms together, so the row could not split it. To cover intervals too short to read, the protocol pre-registers a **repeated-batch, warm** secondary. Every N and Naa process, after its cold interval is recorded, rebuilds setup 20 times and reruns the point's `tauDlp` 3 times. The relations must repeat, and the process reports the mean. This secondary never decides.

Frozen Tier 1 design

The run uses one GitHub `ubuntu-24.04` runner and the baseline run's Sage image, pinned by digest. Every row is one fresh Sage process on one pinned CPU. The native library is built and tested on the runner with Rust 1.99.0 before the schedule starts.

Arm	Role
`B`	**baseline**: unmodified `tau.sage` (Sage Groebner per shift)
`N`	**candidate**: `tauDlp` with `tauRelationNative`
`Naa`	A/A: a second process of N
`NX`	planted positive control: N plus every shift system rebuilt and solved by Sage, compared with the native variety as a set (abort on difference), discarded
`rho`	matched signed-Frobenius rho of rho-arm-20261014 (64 reps per point); S unit and gap only

- **Panel.** 16 new scalars, role `n19-native-heldout-v1`, built by `make_panel.py`. None is an archived scalar, the negative of one, or a $\pm\lambda\hat{}e$ image of one or of another panel scalar. Each point has its own `tauDlp` rng seed, shared by B, N, Naa and NX, so all four draw the same targets.
- **Schedule.** S0 runs first: the Sage variety equals the native variety on 300 systems. O1 follows: the candidate on the ten archived scalars must reproduce run 38520305068. Then come P00–P15, one segment per point, with the five arms in `random.Random(2026101919 + i)` order. A failed S0 or O1 stops the run before any timed segment. The time guard is 300 minutes and the job limit 350.

Primary (pre-registered): the end-to-end cold ratio.

- R = mean cold CPU(B) / mean cold CPU(N) over the 16 points, where cold = setup + online.
- Paired bootstrap over the points, 10,000 draws, seed 20261019 (`stats.py`).
- **Improvement** iff the 95% lower bound exceeds 1; regression iff the upper bound is below 1; otherwise not separated.
- **Invalid** if any gate or control fails, or any scheduled row is missing.

Controls.

- A/A: mean cold N / Naa must lie inside [0.95, 1.05].
- Planted: NX / N must have its 95% lower bound above 1.

Gates.

- Every row ran, exited 0 and verified $s \cdot P = Q$.
- S0 and O1 pass, as above.
- Per point, B, N, Naa and NX agree on: targets, `gb_calls`, `spurious`, relations and log; the relation rows (sha256); and the decomposition multisets (sha256).
- N, Naa and NX produce identical ordered decompositions.
- NX checks every shift system (checked calls = `gb_calls`).
- The generator equals the archived P.
- rho verifies 64 of 64 reps.
- The warm batch repeats the relations.
- Every process runs on one CPU, and every native process loads a library with the same digest.

Power (`power_check.py`, `analysis/power.json`). The prior is the ten archived baseline costs. The noise is a deliberately pessimistic $\sigma = 0.10$ in log per process. With 16 points, power is 1.00 at a true ratio of 1.20 and 0.78 at 1.10. The rejection rate with no effect is 0.033.

Secondaries (never decide).

- Online, setup, wall and launcher-wall ratios. The launcher wall includes Sage start-up.
- Solve time per shift system for each arm.
- N's phase shares.
- `S` per arm, cold, online and setup, in the row's unit (rho CPU per group operation, this run).

- The cold gap of each arm to the matched rho.
- The warm repeated-batch means.

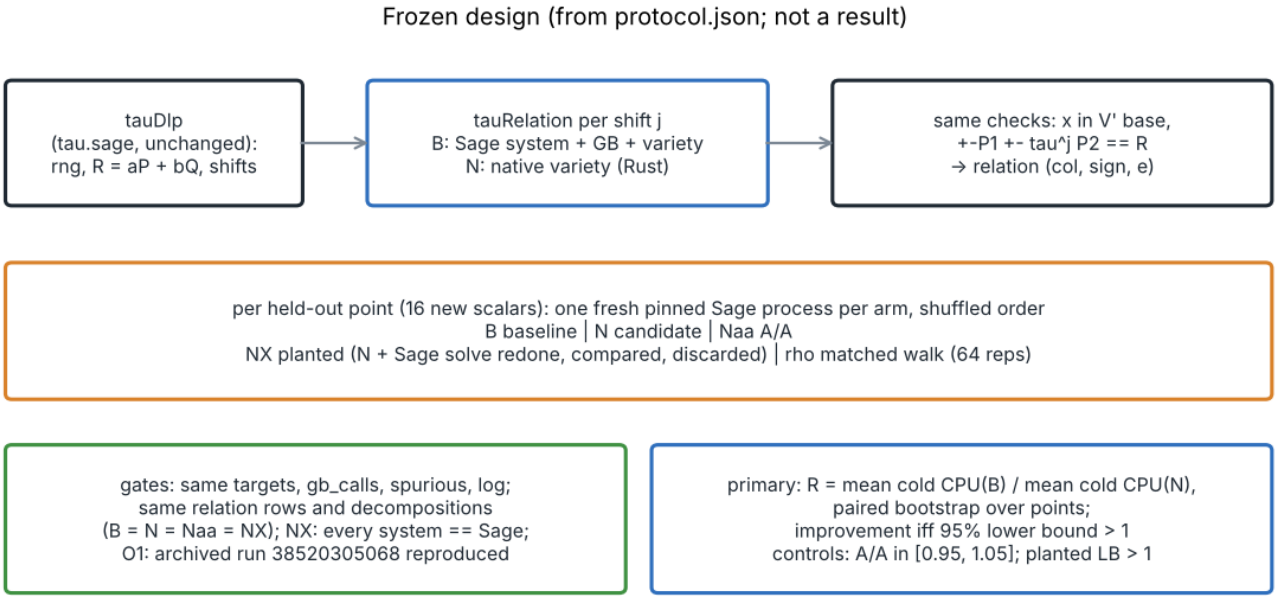


Figure 2: Frozen design (not a result). Source: make_figures.py; vector: figures/design.svg.

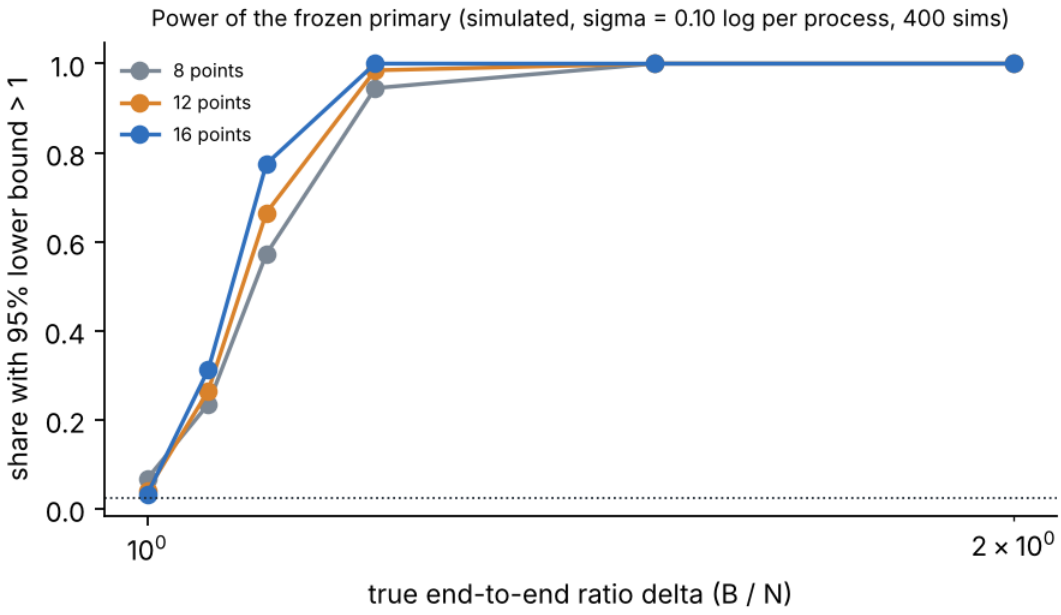


Figure 3: Power of the frozen primary (simulation). Vector: figures/power.svg.

IC1 candidate manifests

The row had none. `inputs/ic1_manifests.json` mints one for the baseline and one for the candidate, using crypto `research/ic_candidate_tournament_20260915/identity.py` (crypto `ecc27ef8`) via `make_ic1_manifest.py`:

- baseline: `IC1N19Ckb0fb1064PDP2polyboriRCsampleLAkernelTDpdpISO0hb41aed7ef80a`;

- candidate: `IC1N19Ckb0fb1064PDP2nativeenumRCsampleLAkernelTDpdpISO0h6952f30ce00e`.

Two caveats:

- **Curve ID.** identity.py derives its own EC1 id from the a2 = 0 model. The row's `EC1N19Ckb1h01617b86df6c` comes from the fb-archive sweep label (E0, b = 1).
- **Columns.** The audited inventory has **28** usable orbit columns, where tau.sage has 29 nominal columns. The V′ point (1, 0) is 4-torsion and Frobenius-fixed, so the cofactor map sends it to O.

What is not claimed

- This is engineering: the same algorithm, the same relations and the same counts. It is not an exponent change.
- The matched rho is still the Python/Sage walk of the baseline row, which is not tuned for wall time. The gap to rho therefore stays a weak reference, as the row already says.
- There is one host class and no replay.

Files

- **Code.**
  - `native/`: the Rust solve and its tests.
  - `arm.sage`: one process per row.
  - `run_schedule.py`: the launcher, with pinning and the time guard.
  - `verify_receipt.py` and `stats.py`: gates, controls, decision and secondaries.
- **Frozen inputs.**
  - `make_protocol.py` and `protocol.json`;
  - `make_panel.py` and `inputs/panel.json`;
  - `make_schedule.py` and `schedule.json`;
  - `power_check.py` and `analysis/power.json`.
- **Workflow.** `.github/workflows/n19-native-tau6.yml`.
- **Local diagnostics.** `smoke/`.
- **Report.** `make_figures.py`, `figures/`, `README.pdf` and `report_header.tex`.